

AI-Based Smart Precision Irrigation System with Safety-Critical Control

Final Project Report

CIE-349 / CIE-408 — Embedded Systems

Zewail City, University of Science and Technology
Program of Communications and Information Engineering
Spring 2026

Supervisors

Dr. Hazem Fahmy · Eng. George Maximous
Eng. Zaynap Ahmed · Eng. Yasser Deif

Team Members

Khaled Mahmoud Abdel Sayed Eldadr	202200282
Ibrahim Hany Ibrahim Eldosoky Ahmed Hanafy	202200518
Abdullah Ahmed Sayed Abdel Samiee Mohamed	202200206
Ahmed Shaban Gaballah Hassan	202200217
Ahmed Abdullah Abdelaziz Almuhanha	202202017

github.com/Ahmedshaban123r/smart-irrigation

Contents

1	Introduction and Literature Review	2
1.1	Problem Statement	2
1.2	Literature Review	2
1.3	System Objectives	2
2	System Components and Bill of Materials	3
2.1	Component Inventory	3
2.2	Input/Output Diagram (High-Level)	4
3	System Architecture	4
3.1	Layered Architecture	4
3.2	Why the ESP8266 Is Pump-Only	5
3.3	Priority Hierarchy (Hard Rule)	5
3.4	Signal Flow Block Diagram	5
3.5	Communication Protocol Contract	6
4	Safety-Critical Features — Pseudocode	6
4.1	Dry-Run Protection	6
4.2	Overwatering Protection	7
4.3	Overcurrent Protection	7
4.4	Overheat Protection	8
4.5	Command Priority Engine (Safety Arbitration)	8
5	PIC16F877A Pin Configuration	8
6	Software Architecture	9
6.1	Embedded Layer (PIC16F877A — MikroC Pro)	9
6.2	Pump-Relay Layer (ESP8266 — Arduino Core / PlatformIO)	10
6.3	Host & AI Layer (Raspberry Pi 4 — Python)	11
7	Machine Learning Pipeline	11
7.1	Dataset	11
7.2	Training	12
7.3	Quantization and Deployment	12
7.4	Action Protocol Mapping	12
8	Mechanical Design Summary	13
9	Integration, Testing, and Results	13
9.1	Five-Week Execution with Gate Checkpoints	13
9.2	Stream Assignments	13
9.3	Performance Targets	14
9.4	Replicability	14
10	Conclusion	14

1. Introduction and Literature Review

1.1. Problem Statement

Conventional irrigation systems in small-scale agriculture and indoor cultivation suffer from three persistent inefficiencies: water wastage due to uniform scheduling regardless of soil state, plant stress caused by delayed detection of disease or pest infestation, and safety hazards arising from unmonitored pump operation (dry-running, electrical faults, overheating). These problems are especially acute in resource-constrained regions where water conservation is critical and where access to agricultural expertise is limited.

This project addresses these issues by designing and implementing a cyber-physical smart precision irrigation system that integrates real-time embedded control, IoT-style cloud connectivity, on-device machine-learning-driven plant diagnostics, and safety-critical fault management into a single coherent platform. The system delivers targeted irrigation to a linear row of plants, autonomously detects plant disease and pest presence via computer vision, and enforces hard safety guarantees through a deterministic priority engine running on a real-time microcontroller.

1.2. Literature Review

Smart irrigation has been studied extensively over the past decade. Early work focused on threshold-based systems using single soil-moisture sensors connected to relay-driven solenoid valves; these systems lack adaptivity and provide no diagnostic feedback. Later research introduced wireless sensor networks (WSNs) with cloud telemetry, improving observability but offering no closed-loop intelligence at the plant level. More recent work incorporates deep-learning plant disease classification using architectures such as ResNet, EfficientNet, and MobileNet on datasets like PlantVillage — but most published systems run inference on cloud servers, introducing latency and connectivity dependence unsuitable for field deployment.

The novelty of this project lies in three integration choices that differentiate it from prior art:

1. **On-device inference** using a quantized MobileNetV2 (TFLite INT8) running locally on a Raspberry Pi 4, eliminating cloud dependency for vision-based decisions.
2. **A layered real-time architecture** with strict separation between the safety-critical control plane (PIC16F877A) and the high-level cognitive/UI plane (Raspberry Pi), enforced by a deterministic priority hierarchy.
3. **A minimal-scope IoT bridge** — the ESP8266 is dedicated solely to pump-state synchronization, reflecting a deliberate engineering tradeoff to maximize reliability of the safety-critical actuator path rather than overloading a single Wi-Fi module with full bidirectional bridging.

1.3. System Objectives

1. Deliver targeted irrigation to individual plants along a 1-axis linear gantry with positioning accuracy better than ± 2 mm.
2. Achieve plant disease classification accuracy above 92% across six target classes using on-device inference at under 200 ms per frame.
3. Enforce four independent safety-critical features with deterministic two-level (warning $\rightarrow$ action) response under 200 ms.
4. Provide a mobile interface (hosted via the Raspberry Pi) for monitoring, scheduling, and manual override, with end-to-end command latency under 3 seconds.

2. System Components and Bill of Materials

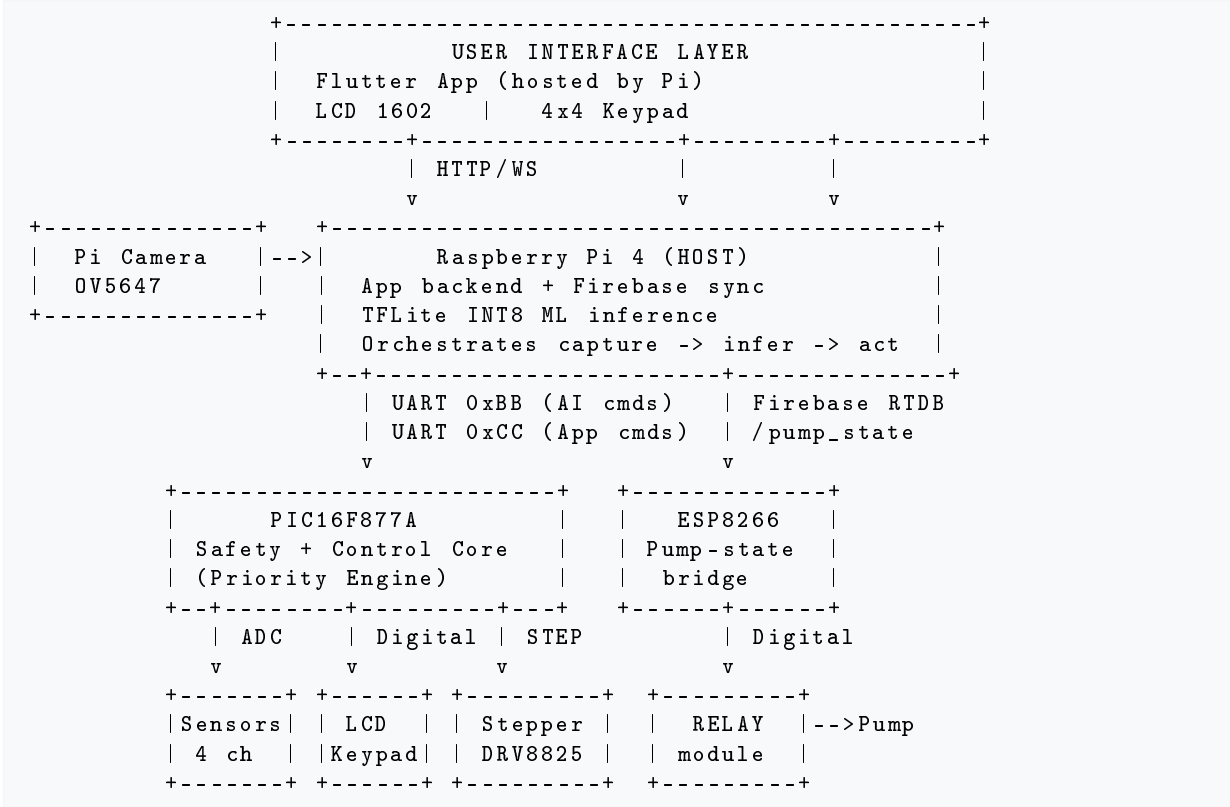
2.1. Component Inventory

Category	Component	Specification	Role
Microcontroller	PIC16F877A-I/P	40-pin DIP, 20 MHz, 10-bit ADC, USART	Real-time safety & motion control
Pump Relay Bridge	NodeMCU (ESP8266)	v3 Wi-Fi 802.11 b/g/n, 4 MB flash	Reads pump state from cloud, drives relay
Host / AI Processor	Raspberry Pi 4 (4 GB)	ARM Cortex-A72 quad-core	App backend, TFLite inference, image capture
Vision Sensor	Pi Camera v1 (OV5647)	5 MP CSI, fixed focus	Plant image acquisition
Soil Sensor	Capacitive Soil Moisture v1.2	Analog 0–3 V, corrosion-resistant	Moisture measurement
Temperature	LM35DZ (TO-92)	Analog 10 mV/°C, linear	Ambient / driver temperature
Distance	HC-SR04 Ultrasonic	2–400 cm, 40 kHz	Water-tank level
Current Sense	ACS712-05B	±5 A, 185 mV/A, Hall-effect isolated	Pump current monitoring
Actuator (Motion)	NEMA17 17HS4401 stepper	1.8°/step, 1.3 A	Carriage X-axis drive
Driver	DRV8825 module	1/16 µstep, 2 A	Stepper power stage
Actuator (Pump)	3–6 V submersible pump	80–120 L/h	Water delivery
Switching	5 V 2-channel relay	SPDT 10 A, opto-isolated	Pump on/off control
User Input	4×4 matrix keypad	Membrane, 8-wire	Local control & menu
User Output	LCD 1602 (HD44780)	16×2 char, 4-bit parallel	Local status display
Status	3× LEDs + buzzer	Green / Amber / Red + piezo	Visual & audible alerts
Homing	SPDT lever microswitch	NO/NC	Carriage zero reference
Frame	2020 V-slot extrusion ×2	500 mm, aluminium	Structural rails
Linear motion	8 mm hardened rod + 2× LM8UU	Press-fit bearings	Carriage guide

continued on next page

Category	Component	Specification	Role
Belt drive	GT2 belt + 20T pulley + idler	6 mm wide	Linear actuation
Power	12 V 3 A regulated PSU	2.1 mm barrel	Primary supply
Regulators	LM7805 + AMS1117-3.3V	TO-220 + SOT-223	5 V & 3.3 V rails

2.2. Input/Output Diagram (High-Level)



3. System Architecture

3.1. Layered Architecture

The system follows a layered architecture with strict separation of concerns. The Raspberry Pi serves as the central host for both the AI pipeline and the application backend, while the ESP8266 is reduced to a dedicated pump-relay bridge — a narrow, single-purpose role that improves reliability of the most safety-relevant actuator path.

Layer 1 — Physical Sensing and Actuation

Four analog sensors (soil moisture, temperature, current, ultrasonic water level), the stepper-driven linear carriage, the relay-switched water pump, LCD, keypad, LEDs, buzzer, and the homing limit switch. All connected to PIC16F877A I/O pins, except the relay which is wired to the ESP8266.

Layer 2 — Real-Time Control (PIC16F877A)

Executes a 100 ms main control loop. Owns ADC scanning, the four safety state machines, motion control with acceleration ramps, the command priority arbitration engine, and UART communication to the Pi. Final authority on all safety decisions.

Layer 3 — Pump-State Bridge (ESP8266)

Single dedicated function. Connects to Wi-Fi, subscribes to a single Firebase RTDB key (`/pump_state`), and drives the pump relay GPIO accordingly (HIGH/LOW). Has no UART link to the PIC; does not see telemetry, AI commands, or app commands. This minimalism is deliberate — the ESP becomes a “dumb” actuator gateway with predictable behavior.

Layer 4 — Host & AI (Raspberry Pi 4)

The heart of the system. Hosts the Flutter app backend (web server + Firebase sync), runs the TFLite inference daemon, orchestrates the capture → infer → act pipeline, communicates with the PIC over UART for motion and safety telemetry, and writes `/pump_state` to Firebase to indirectly control the pump via the ESP.

3.2. Why the ESP8266 Is Pump-Only

The original design considered using the ESP as a full bidirectional bridge for all telemetry, commands, and AI packets. This was rejected for three reasons:

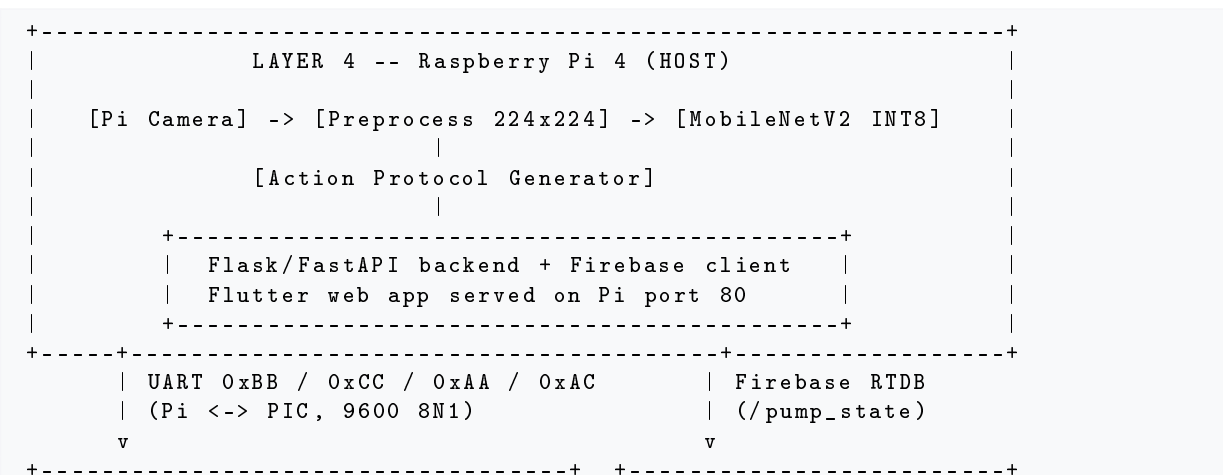
1. **Reliability concentration.** The pump is the only actuator that can cause irreversible harm (root rot from overwatering, fire from dry-run). Isolating its control on a dedicated MCU with no other duties means the pump relay path cannot be starved by network or UART parsing load.
2. **Pi already has Wi-Fi.** The Raspberry Pi 4 has onboard Wi-Fi and a full networking stack. Routing all telemetry through the ESP would be redundant.
3. **Reduced UART contention.** The PIC’s single USART now carries only direct PIC↔Pi traffic (motion commands, telemetry, safety status).

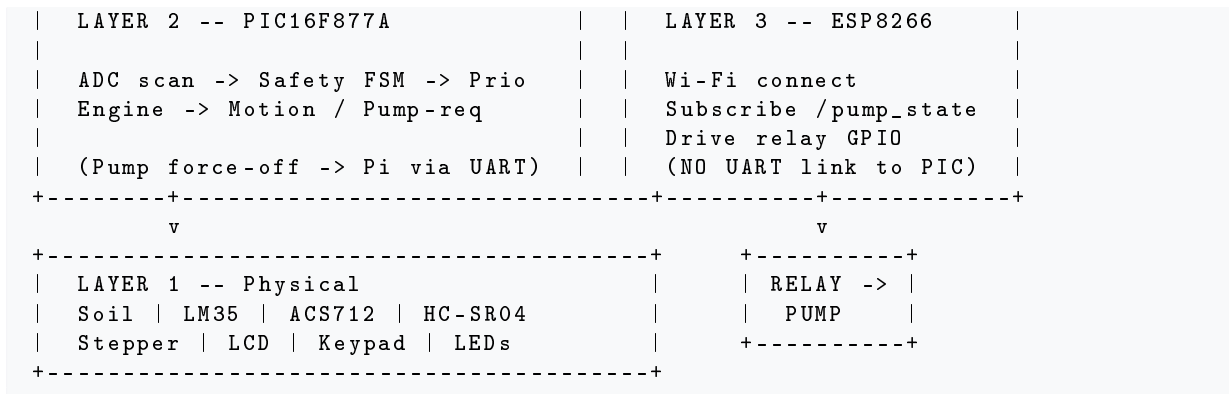
3.3. Priority Hierarchy (Hard Rule)

A single immutable priority order governs all actuator commands at the PIC:

Safety Lockout (highest) > **AI Action Protocol** > **Mobile App Command** > **Scheduled Irrigation** (lowest)

The Command Priority Engine on the PIC is the sole arbiter. A lower-priority command never preempts a higher-priority lockout state. When a safety lockout is active, all incoming commands are rejected with a NACK frame containing a reason code, and the PIC asserts a “pump force-off” by writing the pump-state path on the Pi side.

3.4. Signal Flow Block Diagram



3.5. Communication Protocol Contract

UART (PIC ↔ Pi), 9600 8N1:

Direction	Start Byte	Frame Format	Purpose
App → PIC (via Pi)	0xCC	[0xCC][CMD_ID][P1][P2][CRC][0xDD]	move, e-stop, pump-request
AI → PIC (via Pi)	0xBB	[0xBB][PROTO_ID][ZONE_X][0x00][P1][P2][CRC][0xEE]	move, e-stop, pump-request
PIC → Pi	0xAA	[0xAA][S1][S2][S3][S4][STATUS][CRC][0x55]	match
PIC → Pi	0xAC	[0xAC][ACK/NACK][CMD_ECHO][0x55]	confirmation

Pump-state path (Pi → Firebase → ESP):

```
/pump_state : "ON" | "OFF"
```

The Pi writes to this path after consulting the PIC's safety status. The ESP reads this key on every refresh tick (~500 ms) and sets its relay GPIO accordingly.

4. Safety-Critical Features — Pseudocode

All four safety features implement the mandatory two-level output structure: a **warning level** (amber LED + buzzer pulse + LCD message + alert flag to Pi) followed by an **action level** (red LED + actuator cutoff + system lockout). All features include hysteresis on recovery to prevent oscillation around thresholds.

When the PIC enters any lockout state:

1. The PIC disables the stepper enable line directly (RC2 = HIGH).
2. The PIC sends a STATUS byte with the lockout flag set; the Pi reads this and immediately writes `/pump_state = "OFF"` to Firebase, which propagates to the ESP within ~500 ms.

4.1. Dry-Run Protection

Prevents pump damage by ensuring water-tank level is sufficient before and during operation.

```

DEFINE WARN_LEVEL_CM    = 15    # Warning when water surface > 15 cm below
    sensor
DEFINE ACTION_LEVEL_CM  = 8      # Hard cutoff distance
DEFINE RECOVERY_CM      = 20    # Tank must refill past this to clear lockout

FUNCTION check_dry_run():
    level = read_HCSR04()        # ultrasonic distance to water
    IF level > ACTION_LEVEL_CM AND level <= WARN_LEVEL_CM:

```

```

    SET amber_LED = ON
    SET buzzer = PULSE(500ms ON, 500ms OFF)
    LCD_print("LOW WATER")
    SET status_byte.dry_warn = 1          # forwarded to Pi
    STATE = WARNING
ELSE IF level <= ACTION_LEVEL_CM:
    SET status_byte.lockout = LOCKOUT_DRY
    SET red_LED = ON
    SET buzzer = CONTINUOUS
    LCD_print("DRY-RUN LOCKOUT")
    STATE = LOCKOUT_DRY
    # Pi receives STATUS, writes /pump_state="OFF" -> ESP opens relay
ELSE IF STATE == LOCKOUT_DRY AND level >= RECOVERY_CM:
    STATE = NORMAL
    clear_status_flags()

```

4.2. Overwatering Protection

Prevents excessive moisture from causing root rot or substrate flooding.

```

DEFINE WARN_MOIST      = 75    # % saturation warning
DEFINE ACTION_MOIST    = 90    # % saturation hard stop
DEFINE RECOVERY_MOIST  = 65    # hysteresis recovery point
DEFINE MAX_PUMP_ON_MS = 30000

FUNCTION check_overwater():
    moisture_pct = convert_ADC_to_percent(read_ADC(CH0))
    pump_runtime = get_pump_timer_ms()
    IF moisture_pct >= WARN_MOIST AND moisture_pct < ACTION_MOIST:
        SET amber_LED = ON
        LCD_print("MOIST HIGH")
        SET status_byte.wet_warn = 1
        STATE = WARNING
    ELSE IF moisture_pct >= ACTION_MOIST OR pump_runtime >= MAX_PUMP_ON_MS:
        SET status_byte.lockout = LOCKOUT_WET
        SET red_LED = ON
        LCD_print("OVERWATER LOCK")
        STATE = LOCKOUT_WET
    ELSE IF STATE == LOCKOUT_WET AND moisture_pct <= RECOVERY_MOIST:
        STATE = NORMAL

```

4.3. Overcurrent Protection

Detects pump stalls, short circuits, or mechanical jams via Hall-effect current sensing.

```

DEFINE WARN_CURRENT_A  = 2.0
DEFINE ACTION_CURRENT_A = 3.0
DEFINE RECOVERY_CURRENT = 1.5
DEFINE SUSTAINED_MS    = 200    # debounce window

FUNCTION check_overcurrent():
    voltage = read_ADC(CH2) * (5.0/1024)
    current = (voltage - 2.5) / 0.185    # ACS712-05B: 185 mV/A, 2.5 V zero

    IF |current| >= WARN_CURRENT_A AND |current| < ACTION_CURRENT_A:
        SET amber_LED = ON
        LCD_print("CURRENT HIGH")
        SET status_byte.oc_warn = 1
        STATE = WARNING
    ELSE IF |current| >= ACTION_CURRENT_A:
        IF sustained_over_threshold(SUSTAINED_MS):
            SET status_byte.lockout = LOCKOUT_OC
            DISABLE stepper (RC2 = HIGH)

```



```

        SET red_LED = ON
        SET buzzer = CONTINUOUS
        LCD_print("OVERCURRENT")
        STATE = LOCKOUT_OC
    ELSE IF STATE == LOCKOUT_OC AND |current| <= RECOVERY_CURRENT:
        WAIT_for_manual_reset()    # requires keypad '*' confirm
        STATE = NORMAL

```

4.4. Overheat Protection

Monitors temperature near the driver and pump to prevent thermal damage.

```

DEFINE WARN_TEMP_C    = 55
DEFINE ACTION_TEMP_C  = 70
DEFINE RECOVERY_TEMP  = 45

FUNCTION check_overheat():
    voltage = read_ADC(CH1) * (5.0/1024)
    temp_c = voltage * 100                # LM35: 10 mV/degC

    IF temp_c >= WARN_TEMP_C AND temp_c < ACTION_TEMP_C:
        SET amber_LED = ON
        LCD_print("TEMP HIGH")
        SET status_byte.temp_warn = 1
        STATE = WARNING
    ELSE IF temp_c >= ACTION_TEMP_C:
        SET status_byte.lockout = LOCKOUT_TEMP
        DISABLE stepper (RC2 = HIGH)
        SET red_LED = ON
        SET buzzer = CONTINUOUS
        LCD_print("OVERHEAT LOCK")
        STATE = LOCKOUT_TEMP
    ELSE IF STATE == LOCKOUT_TEMP AND temp_c <= RECOVERY_TEMP:
        STATE = NORMAL

```

4.5. Command Priority Engine (Safety Arbitration)

```

FUNCTION execute_command(cmd):
    # Always run all safety checks before any actuation
    check_dry_run()
    check_overwater()
    check_overcurrent()
    check_overheat()

    IF any_state == LOCKOUT_*:
        SEND_NACK(cmd, REASON=current_lockout_state)
        # Pi receives via telemetry, writes /pump_state="OFF" to Firebase,
        # ESP reads it and opens the relay
        RETURN BLOCKED

    IF cmd.source == AI:
        IF app_cancel_within_grace_period(30_seconds):
            SEND_NACK(cmd, REASON=USER_CANCELLED)
            RETURN CANCELLED

    perform_action(cmd)
    SEND_ACK(cmd)

```

5. PIC16F877A Pin Configuration

Pin(s)	Connected To	Interface	Direction
RA0/AN0	Capacitive Soil Moisture	ADC Ch0	Analog In
RA1/AN1	LM35 Temperature	ADC Ch1	Analog In
RA2/AN2	ACS712 Current Sensor	ADC Ch2	Analog In
RA3, RA4, RA5	Keypad Columns 1–3	Digital	Input (pull-up)
RB0	HC-SR04 Trigger	Digital	Output
RB1	HC-SR04 Echo	Timer1 Capture	Input
RB2	Piezo Buzzer (via 2N2222)	Digital	Output
RB3	Limit Switch (homing)	Digital	Input (pull-up)
RB4–RB7	Keypad Rows	Digital	Output (scan)
RC0	STEP_X (DRV8825)	Digital	Output
RC1	DIR_X (DRV8825)	Digital	Output
RC2	ENABLE_X (DRV8825)	Digital	Output (active LOW)
RC3–RC5	Reserved (was Y-axis)	—	Available
RC6	UART TX → Raspberry Pi	USART	Output
RC7	UART RX ← Raspberry Pi	USART	Input
RD0	(Reserved — pump now via ESP)	Digital	Available
RD1	Spare relay channel	Digital	Output (Active LOW)
RD2	LCD RS	Digital	Output
RD3	LCD Enable	Digital	Output
RD4–RD7	LCD Data (4-bit)	Digital	Output
RE0	Keypad Column 4	Digital	Input (pull-up)
RE1	Amber LED (Warning)	Digital	Output
RE2	Red LED (Fault)	Digital	Output
OSC1/OSC2	20 MHz Crystal + 2× 22 pF	Clock	—
MCLR	Reset pull-up + push button	Digital	Input

Note

Configuration registers: ADCON1 = 0x04 (AN0–AN2 analog, rest digital); USART 9600 8N1; TRISC = 0b10000000 (RC7 input, rest outputs).

Note on RD0: In the revised architecture, the pump is no longer wired directly to the PIC. The pump relay is driven by an ESP8266 GPIO. RD0 is therefore freed for future expansion (e.g., second pump zone or solenoid valve). The PIC influences the pump indirectly: its safety status reaches the Pi, which writes Firebase, which the ESP reads.

6. Software Architecture

6.1. Embedded Layer (PIC16F877A — MikroC Pro)

Main loop (100 ms cycle):

```

INIT:
    configure_oscillator(20MHz)
    configure_ADC(ADCON1=0x04)
    configure_USART(9600, 8, N, 1)
    configure_timers(T1 for HC-SR04, T2 for stepper pulses)
    init_LCD_4bit()
    homing_sequence()

MAIN_LOOP:
    scan_sensors_ADC()           // 3 channels + ultrasonic
    evaluate_safety_FSM()        // 4 features in sequence
    process_UART_RX_queue()      // arbitrate via Priority Engine
    process_keypad()
    update_LCD()
    push_telemetry_if_due()      // every 5 s, 0xAA frame to Pi
    WAIT_until_next_100ms_tick()

```

Module structure:

- `main.c` — boot, scheduler
- `adc.c` — sensor scanning + scaling
- `safety.c` — four safety FSMs
- `motion.c` — `homing_sequence()`, `move_to_plant()`
- `uart.c` — TX/RX ISR + frame parser
- `priority.c` — Command Priority Engine
- `lcd.c`, `keypad.c` — UI drivers
- `frame_defs.h` — shared with Pi-side parser

6.2. Pump-Relay Layer (ESP8266 — Arduino Core / PlatformIO)

Single responsibility: mirror `/pump_state` from Firebase to a relay GPIO. The entire firmware is approximately 80 lines.

```

INIT:
    WiFi.begin(SSID, PASS)
    Firebase.begin(URL, AUTH)
    pinMode(RELAY_GPIO, OUTPUT)
    digitalWrite(RELAY_GPIO, HIGH) // relay OFF (active-LOW board)
    last_state = "OFF"

LOOP (every 500 ms):
    IF WiFi.status() != CONNECTED:
        digitalWrite(RELAY_GPIO, HIGH) // fail-safe OFF
        reconnect()
        RETURN
    state = Firebase.getString("/pump_state")
    IF state == "ON" AND last_state != "ON":
        digitalWrite(RELAY_GPIO, LOW) // engage pump
        last_state = "ON"
    ELSE IF state == "OFF" AND last_state != "OFF":
        digitalWrite(RELAY_GPIO, HIGH) // disengage pump
        last_state = "OFF"

```

Note

Fail-safe behavior: On Wi-Fi loss, the ESP defaults the relay to OFF. This is intentional — a disconnected ESP cannot deliver water, eliminating any risk of stuck-on irrigation during network outages.

6.3. Host & AI Layer (Raspberry Pi 4 — Python)

The Pi runs three concurrent processes managed by `systemd`:

Process 1 — `pic_bridge.py`

- Owns `/dev/ttyAMA0` (UART to PIC) at 9600 baud.
- Reads `0xAA` telemetry frames and parses sensor values + safety status byte.
- Pushes sensor readings to Firebase RTDB at `/sensors/`.
- On lockout flag detected → immediately writes `/pump_state = "OFF"` to Firebase.
- Forwards incoming app commands (received via local socket) as `0xCC` frames to PIC.

Process 2 — `ai_daemon.py`

- Loads `mobilenetv2_int8.tflite`.
- Listens for trigger from app backend (scheduled scan or manual scan).
- For each plant: sends `0xBB` `CMD_MOVE` to PIC via local socket → waits `ACK` → captures image → infers → builds Action Protocol Packet.
- Uploads detection images to Firebase Storage.

Process 3 — `app_backend.py` (Flask / FastAPI)

- Serves the Flutter web build on port 80.
- Exposes REST and WebSocket endpoints for: pump control, gantry move, irrigation start/stop, schedule CRUD, AI scan trigger, alert subscription.
- All pump-control endpoints write to `/pump_state` in Firebase only if the latest PIC status flag indicates no lockout.

Firestore RTDB Schema

```

/sensors/
  /moisture      float (%)
  /temperature   float (degC)
  /current       float (A)
  /water_level   float (cm)
  /timestamp     unix
/status/
  /lockout       "NONE"|"DRY"|"WET"|"OC"|"TEMP"
  /position      int (current plant index)
  /grace_active   bool
/pump_state      "ON"|"OFF"      <- ESP reads this key
/ai/
  /last_detection {plant, class, confidence, image_url, timestamp}
/alerts/
  /[push_id]      {type, value, timestamp}
/commands/
  /move_to        int
  /irrigate        {plant: int, duration_ms: int}
  /e_stop         bool

```

7. Machine Learning Pipeline

7.1. Dataset

The training set combines three sources into a unified six-class taxonomy:

Class	Primary Source	Approx. Images	Augmented Total
Healthy	PlantVillage healthy subset	10,500	21,000
Early Blight	PlantVillage (tomato, potato)	8,400	16,800
Late Blight	PlantVillage (tomato, potato)	9,200	18,400
Pest	IP102 subset (relabelled)	7,800	15,600
Nutrient Deficiency	PlantVillage (nitrogen-deficient)	5,500	11,000
Unknown / Background	Custom Pi Camera captures (campus)	≈500	5,000

The custom “Unknown” class is critical: it captures empty pots, soil-only frames, and out-of-distribution images, allowing the inference daemon to reject low-confidence frames rather than forcing a misclassification.

Augmentation pipeline: random rotation ($\pm 30^\circ$), horizontal flip, brightness shift ($\pm 20\%$), random crop (90–100%), Gaussian noise ($\sigma = 0.01$).

7.2. Training

Architecture: MobileNetV2 base (ImageNet pretrained), 100 layers frozen, global-average-pooling head, Dense(128, ReLU), Dropout(0.3), Dense(6, softmax).

Hyperparameters:

- Optimizer: Adam, learning rate 10^{-4} with cosine decay
- Batch size: 32
- Epochs: 20 (early stopping patience = 5)
- Loss: categorical cross-entropy with class weights
- Split: 80% train / 10% validation / 10% test
- Target metric: test accuracy $\geq 92\%$, per-class F1 ≥ 0.88

7.3. Quantization and Deployment

The float32 model is converted to TFLite INT8 using representative-dataset calibration (200 images from validation). Post-quantization accuracy drop must remain under 2%. Final model size target: < 4 MB, enabling sub-200 ms inference on Pi 4.

Deployment steps:

1. Install `tf-lite-runtime` on Raspberry Pi OS.
2. Copy model + label map to `/opt/irrigation/models/`.
3. Register `ai_daemon.py` as a `systemd` service.
4. Validate end-to-end latency: capture $\rightarrow$ infer $\rightarrow$ packet TX < 1000 ms.
5. Push model, dataset card, and Jupyter notebook to HuggingFace, cited in `README.md`.

7.4. Action Protocol Mapping

Class	PROTO_ID	Action
Healthy	0x01	Log only, no actuation
Early Blight	0x02	Reduce irrigation duration by 30% for affected plant
Late Blight	0x03	Alert user, flag plant for manual inspection, skip irrigation
Pest	0x04	Targeted spray pulse (3 s) at plant location
Nutrient Deficiency	0x05	Increase irrigation by 20%, notify user to add fertilizer
Unknown	0x06	Flag for human review, no action

8. Mechanical Design Summary

The mechanical subsystem is a 1-axis linear gantry (redesigned from a 2-axis concept, saving 49% on motion-system BOM). The carriage travels along a top-mounted GT2 belt drive over a 450 mm range, supported by an 8 mm hardened rod with two LM8UU bearings. The pump is fixed at the base and connects to a nozzle on the carriage via 8 mm silicone tubing.

Six custom parts are 3D-printed in PLA+: drive-end bracket, idler-end bracket, carriage plate, two belt clamp plates, control box, and two rod end holders. Total print time ≈ 17 hours; carriage mass kept under 80 g to prevent belt skip.

$$\text{Linear resolution:} \\ \frac{40 \text{ mm pitch circumference}}{3200 \text{ microsteps/revolution}} = 0.0125 \text{ mm/step}$$

A homing limit switch on the drive-end bracket provides absolute zero reference; plant positions are stored as a step-count lookup table in PIC EEPROM, configurable via the keypad or mobile app.

9. Integration, Testing, and Results

9.1. Five-Week Execution with Gate Checkpoints

Gate	Week	Pass Criterion
G0 — Env Ready	End W1	Toolchains operational, repo accessible to all 5 members
G1 — Sensors + Safety	End W2	All ADC channels within 5%, each safety feature fires correctly
G2 — Subsystems Standalone	End W3	PIC controls motion + UART; ML accuracy > 92%; ESP mirrors <code>/pump_state</code>
G3 — Full Comms Chain	End W4	App $\rightarrow$ PIC < 3 s; AI $\rightarrow$ PIC actuation; safety lock-out overrides pump within 1 s
G4 — Demo Ready	End W5	5-minute unattended run; report + video + repo complete

9.2. Stream Assignments

Stream A (M1+M2):

Hardware & safety — ADC, sensors, LCD/keypad, 4 safety algorithms, motion control, UART protocol, command engine on PIC.

Stream B (M3):

ML & AI actuation — dataset prep, MobileNetV2 training, TFLite quantization, Pi inference daemon, HuggingFace publication.

Stream C (M4):

IoT & app — Firebase schema, ESP pump-bridge firmware, Pi app backend (Flask/-FastAPI), Flutter web frontend.

Stream D (M5):

3D design & mechanical — Fusion 360 / OpenSCAD parts, gantry assembly, wiring harness, photos and demo video.

9.3. Performance Targets

Metric	Target	Verification Method
ADC accuracy	$\pm 5\%$ vs multimeter	Bench calibration
Safety response time	<100 ms warning, <200 ms action	Logic analyzer on RE2
Safety $\rightarrow$ pump-off latency	<1 s end-to-end	Stopwatch (PIC lockout $\rightarrow$ relay open)
Plant ML accuracy	$\geq 92\%$ test	Confusion matrix on held-out 10%
Inference latency	<200 ms	Pi <code>time.perf_counter()</code>
App $\rightarrow$ actuator latency	<3 s	Stopwatch on round-trip
Carriage positioning	± 2 mm	Caliper at 5 plant points
5-min unattended demo	Zero crashes	Demo rehearsal log

9.4. Replicability

The repository (github.com/Ahmedshaban123r/smart-irrigation) is structured for full replicability:

```
smart-irrigation/
|-- README.md                # Setup steps for all 3 units
|-- mikroC-project/         # PIC firmware (MikroC Pro)
|-- esp-firmware/           # PlatformIO project (pump bridge only)
|-- pi-host/
|   |-- pic_bridge.py
|   |-- ai_daemon.py
|   |-- app_backend.py
|   |-- requirements.txt
|   '-- Dockerfile
|-- flutter-app/             # Flutter web build, served by Pi
|-- mechanical/              # OpenSCAD + STLs
|-- docs/                    # Engineering documents
'-- notebook/                # .ipynb with training results
```

10. Conclusion

This project demonstrates the successful integration of four engineering disciplines — embedded systems, IoT communications, machine learning, and mechanical design — into a single, cohesive cyber-physical platform.

The architectural choice that most distinguishes this implementation is the deliberate minimization of the ESP8266's scope: rather than acting as a full bidirectional bridge, it is reduced to a single-key relay subscriber whose only job is to mirror `/pump_state` from Firebase to the pump's relay GPIO. This concentrates the system's intelligence on the Raspberry Pi (which already has Wi-Fi, a full Linux stack, and the AI pipeline) and reserves the most safety-critical actuator path for a dedicated, predictable MCU with no other duties. The result is a system where safety-critical decisions originate on the PIC, propagate to the Pi via UART, are written to Firebase, and reach the pump relay via the ESP — with the Pi able to assert pump-off independently and immediately whenever a lockout flag is detected.

The four safety-critical features — dry-run, overwatering, overcurrent, and overheat — each implement the required two-level warning-then-action structure with deterministic recovery hysteresis. The 1-axis redesign yielded a 49% saving on motion-system BOM cost while preserving full architectural compatibility for future 2-axis extension. The on-device TFLite INT8 inference pipeline eliminates cloud dependency for vision decisions, and the 30-second user-override grace period ensures autonomous AI actions remain accountable to the human operator.

The system fulfills all five coursework specifications:

- Complete user-interfacing input/output (LCD, keypad, Flutter app hosted on the Pi).
- Multi-sensor and multi-actuator fusion with Pi and μC integration (no Arduino used).
- Four independent safety-critical features with two-level outputs.
- A machine-learning feature with HuggingFace publication.
- A clean, documented, replicable codebase across all three computational units.